PNGtoCC3Background

User Manual

Creating wide CoCo 3 scrolling backgrounds

PNGtoCC3Background version 1.00
BasTo6809 compiler version 5.46
Glen Hewlett - August 2026

Contents

What PNGtoCC3Background does

PNGtoCC3Background converts one wide PNG picture into a sequence of overlapping 512-pixel-wide CoCo 3 background windows. Each pixel is reduced to one of 16 palette colours and packed two pixels per byte.

Its stock BasTo6809 use is a horizontally scrolling **PLAYFIELD 5** scene:

- the visible screen is 256 x 192 pixels;
- each stored window is 512 x 192 pixels;
- successive windows advance 256 pixels, so adjacent windows overlap; and
- the converted data starts at CoCo 3 physical 8K block 128 (`$80`).

The default output is a numbered set of ordinary DECB LOADM files. The `-big` option instead creates one special file for `SDC_BIGLOADM` .

> **Hardware requirement:** the standard compiler workflow uses physical RAM above 1 MB and therefore requires a 2 MB CoCo 3. This is not a CoCo 1/2 image format.

For vertical playfields or other fixed playfield shapes, use the separate PNGtoCC3Playfield utility.

Five-minute tutorial

1. Prepare the picture

Create an opaque PNG named `MAP.png` with these recommended dimensions:

```
Width:  512 to 2560 pixels, in steps of 256
Height: exactly 192 pixels
```

PNGtoCC3Background reads RGB colour but does not use PNG transparency. Flatten transparent artwork onto the intended background colour before converting it.

Place `MAP.png`, `PNGtoCC3Background`, and `CoCo3_Palette.asm` in the same folder. The current program removes directory information from the input name before loading it, so this same-folder workflow is the reliable one.

2. Create the project palette once

For the first representative image in a project, run:

```
./PNGtoCC3Background –makepal MAP.png
```

This creates or replaces `CoCo3_Palette.asm`, chooses 16 commonly used CoCo 3 colours from the picture, and converts the background with that palette.

Keep that palette. Convert every later background and sprite without `–makepal` so that all assets use the same 16 colour indices.

3. Choose an output format

Ordinary numbered LOADM files are the default:

```
./PNGtoCC3Background MAP.png
```

A 1024 x 192 picture creates:

```
MAP00.BIN
MAP01.BIN
MAP02.BIN
```

For a CoCoSDC, one streaming file is more convenient:

```
./PNGtoCC3Background –big MAP.png
```

This creates only `MAP00.BIN`, in the special `SDC_BIGLOADM` format.

4. Load the result

For normal output, load every numbered file in order:

```
LoadM "MAP00.BIN"
LoadM "MAP01.BIN"
LoadM "MAP02.BIN"
```

The drive suffix may be included when loading from floppy:

```
LoadM "MAP00.BIN:0"
```

On the CoCoSDC, either load the numbered files:

```
SDC_LoadM "MAP00.BIN", 0
SDC_LoadM "MAP01.BIN", 0
SDC_LoadM "MAP02.BIN", 0
```

or load the one `-big` file:

```
SDC_BigLoadM "MAP00.BIN", 0
```

Do not use `LOADM` or `SDC_LOADM` on a file made with `-big`.

Preparing the PNG

Dimensions

Version 1.00 reports that widths of 256 pixels are accepted, but its window writer needs at least 512 pixels. Use these practical limits:

Property	Supported project value
Width	512 through 2560 pixels
Width step	256 pixels
Height for PLAYFIELD 5	192 pixels
Colours on the CoCo	16
Stored pixels per byte	2

The program truncates width down to a multiple of 256 and height down to a multiple of 64. It does not scale the picture. For example, a 768 x 200 PNG is converted as 768 x 192; the bottom eight rows are discarded.

Although the converter accepts taller images, the stock PLAYFIELD 5 compiler routines are designed for 192-pixel-high horizontal backgrounds. Use PNGtoCC3Playfield for the other supported layouts.

Transparency and colour

The input may be a 32-bit RGBA PNG, but alpha is ignored. A transparent pixel is converted from the RGB value stored underneath its alpha channel. This can produce unexpected colours around transparent artwork.

Before conversion:

- flatten the image onto its final background colour;
- remove soft transparent edges;
- use hard pixel edges where possible; and
- preview the converted result in MAME or on the intended display.

File names and location

Use a simple base name with letters, numbers, and underscores:

```
FOREST.png
LEVEL_01.png
SPACE_MAP.png
```

Avoid spaces, punctuation, and a base name so long that the CoCo storage system cannot preserve it. Output is written beside the utility and is named from the PNG base name.

Palette workflow

Creating the palette

`-makepal` quantizes the source RGB values to CoCo 3 colour values, finds the 16 most frequent results, and writes them as assembly bytes:

```
CoCo3_Palette.asm
```

This is a frequency-based choice, not a perceptual colour-clustering system. A palette made from one unusually dark or monochrome image may be poor for the rest of a game. A title screen, representative level, or prepared palette reference image is usually a better source.

Reusing the palette

After creating it once, omit `-makepal`:

```
./PNGtoCC3Background -d1 LEVEL_02.png
```

The utility reads the first 16 `FCB %xxxxxxxx` values from the existing file and maps every pixel to the nearest entry.

Use the identical `CoCo3_Palette.asm` for:

- every background in the scene;
- every CoCo 3 sprite drawn over it; and
- the compiled program that installs the hardware palette.

If the program contains a CoCo 3 `SPRITE_LOAD`, BasTo6809 includes and installs the shared palette with the sprite support. A background-only program must install all 16 entries itself with `PALETTE` commands or equivalent assembly code.

> Regenerating the palette changes the meaning of every four-bit pixel index. If the palette changes, reconvert every asset that uses it.

Understanding the output

Overlapping windows

Each file or window represents 512 source pixels. The starting source X positions are:

```
0, 256, 512, 768, ...
```

The overlap lets a 256-pixel viewport cross a boundary without needing to assemble two unrelated image rows during display.

For a converted width `W`:

```
number of windows = W / 256 - 1
```

At 192 pixels high, each window contains 49,152 packed image bytes and occupies six 8K blocks.

Planning table

The stock compiler expects `-blk128`:

PNG size	Windows/files	Image bytes	Physical blocks
512 x 192	1	49,152	`$80-$85`
768 x 192	2	98,304	`$80-$8B`
1024 x 192	3	147,456	`$80-$91`
1536 x 192	5	245,760	`$80-$9D`
2560 x 192	9	442,368	`$80-$B5`

An ordinary 512 x 192 output file is 49,229 bytes because DECB segment headers and the postamble are stored in addition to the pixels.

The default bank `$80` begins at physical address `$100000`. A maximum 2560 x 192 map ends at `$16BFFF`. The converter does not check for collisions with sprites, audio, movies, or other extended-memory data.

Output formats: each default numbered file is an ordinary DECB stream that maps a destination block into `$4000`, loads its 8K payload, and restores the normal MMU mapping. Load every file in ascending order with `LOADM` or `SDC_LOADM`.

One `-big` file instead contains a 512-byte physical-block table followed by packed 8K payloads. Load it only with `SDC_BIGLOADM`.

Using it with BasTo6809

Required graphics layout

The converted layout matches:

```
Playfield 5
Gmode 136, 0
```

GMODE 136 is the 256 x 192, 16-colour CoCo 3 screen. PLAYFIELD 5 describes the horizontally scrolling bank layout produced by this converter.

Leave the starting block at its default:

```
./PNGtoCC3Background –blk128 MAP.png
```

The compiler's stock PLAYFIELD 5 code assumes block `$80`. A different `–blkN` value requires custom assembly changes; it is not a general relocation option for an ordinary compiled program.

Scrolling coordinates

After loading, `VIEW x,y` selects the top-left corner of the visible 256 x 192 window. For a 1024 x 192 map:

```
View 0, 0
View 128, 0
View 512, 0
View 768, 0
```

Keep X between 0 and `image width – 256`. PLAYFIELD 5 does not provide vertical travel, so Y normally remains zero. The hardware path scrolls horizontally in four-pixel steps; the low two bits of X do not change the displayed position.

Current update requirement

In compiler version 5.46, `VIEW` calculates the new playfield values, while the CoCo 3 sprite `WAIT VBL` path commits those values to the GIME registers. The intended game workflow therefore uses playfield sprites and calls `WAIT VBL` after changing the view.

For sprites on PLAYFIELD 5, generate them with PNGtoCCSB's `–scroll` option:

```
./PNGtoCCSB –g136 –scroll PLAYER.png
```

A pure background viewer with no sprite system must currently supply equivalent register-update assembly. The present double-buffer update path is not a fully generic playfield display path, so verify the selected map and buffer behavior in MAME and on the target hardware.

Compiler outline

This outline shows the relationship between the commands; substitute a real scrolling sprite and its update loop:

```
'compileoptions -s16 -i

Sprite_Load "PLAYER_SCROLL.asm", 0

Playfield 5
Gmode 136, 0

SDC_BigLoadM "MAP00.BIN", 0

Screen 1, 0
View 0, 0

' Queue the sprite work used by the playfield update system.
Sprite Locate 0, 20, 80
Sprite Backup 0
Sprite Show 0, 0
Wait VBL
```

Dithering choices

Option	Method	Best use
`-d0`	Nearest palette colour	Crisp pixel art, flat colours, fastest conversion
`-d1`	Floyd-Steinberg	General pictures and gradients; default
`-d2`	Blue-noise-style randomized mask	Textured results with less directional patterning
`-d3`	Ordered pattern	Stable, regular dither suited to some retro artwork

Dithering cannot add colours beyond the 16-entry palette. It arranges existing palette colours so that an area appears closer to the source when viewed at normal size.

For pixel art, begin with `-d0`. For painted or photographic material, compare `-d1` and `-d3` on the target display.

Command-line reference

```
PNGtoCC3Background [-dN] [-makepal] [-blkN] [-big] INPUT.png
```

All options are case-insensitive and must appear before the PNG filename.

Option	Default	Meaning
`-d0`		No dithering; choose the nearest palette entry
`-d1`	selected	Floyd-Steinberg dithering
`-d2`		Blue-noise-style dithering
`-d3`		Ordered dithering
`-makepal`	off	Create or replace `CoCo3_Palette.asm`, then use it
`-blkN`	`-blk128`	Starting physical 8K block, written as a decimal number
`-big`	off	Make one special `SDC_BIGLOADM` file

Correct:

```
./PNGtoCC3Background -d0 -blk128 -big CITY.png
```

Incorrect:

```
./PNGtoCC3Background CITY.png -big
./PNGtoCC3Background -blk80 CITY.png
```

The first puts an ignored option after the filename. The second means decimal 80, not hexadecimal `$80`; use `-blk128` for physical block `$80`.

Troubleshooting

No output file is created

- Use a width of at least 512 pixels; version 1.00 writes no window for a 256-pixel-wide image.
- Put a readable, nonempty PNG beside the utility, run there, and put every option before the filename.

The utility cannot find the palette

Run once with `-makepal`, or put the project's established `CoCo3_Palette.asm` beside the utility. Replace any zero-byte palette left by an earlier failed attempt.

Colours are wrong

- Install the same `CoCo3_Palette.asm` used for conversion, and reconvert every asset after changing it.
- Flatten transparency and confirm sprites use that same palette.

The image is cropped

The program truncates dimensions; it does not resize. Make the source width an exact multiple of 256 and use a 192-pixel height for PLAYFIELD 5.

Scrolling shows the wrong memory

- Use a 2 MB CoCo 3.
- Convert with `-blk128` for the stock compiler.
- Load every numbered file in order, or load the `-big` file with `SDC_BIGLOADM`.
- Use PLAYFIELD 5 and GMODE 136.
- In the current compiler, perform the sprite/`WAIT VBL` update that commits the view values.

A normal loader rejects the `-big` file

This is expected. A `-big` file is not a DECB LOADM stream. Load it only with:

```
SDC_BigLoadM "MAP00.BIN", 0
```

Version 1.00 can list more physical blocks in a `-big` table than it writes for an overlapping background. The present SDC loader stops at physical end-of-file, but the metadata is inconsistent. Verify `-big` output on the target system; ordinary numbered files are the conservative choice.

Project checklist

- ☐ The target is a 2 MB CoCo 3.
- ☐ The PNG is opaque, at least 512 pixels wide, and exactly 192 pixels high.
- ☐ The width is a multiple of 256 and no greater than 2560 for stock PLAYFIELD 5.
- ☐ One representative image created the shared palette with `-makepal`.
- ☐ Every later image and sprite reused that exact palette.
- ☐ The converter used the stock starting block, decimal `-blk128`.
- ☐ Normal numbered files are loaded in order, or the one `-big` file uses `SDC_BIGLOADM`.
- ☐ The BASIC program selects PLAYFIELD 5 and GMODE 136.
- ☐ Scrolling sprites were converted with PNGtoCCSB `-scroll`.
- ☐ The sprite/`WAIT VBL` path commits each changed `VIEW` position.
- ☐ The final result was tested in MAME and on the intended hardware.

Command examples

Create a shared palette and ordinary output:

```
./PNGtoCC3Background -d1 -makepal FOREST.png
```

Reuse that palette with crisp nearest-colour matching:

```
./PNGtoCC3Background -d0 FOREST_NIGHT.png
```

Create a single CoCoSDC streaming file:

```
./PNGtoCC3Background -d1 -blk128 -big CITY.png
```

Load three normal files from floppy drive 0:

```
LoadM "CITY00.BIN:0"
LoadM "CITY01.BIN:0"
LoadM "CITY02.BIN:0"
```

Load the single streaming file from CoCoSDC slot 1:

```
SDC_BigLoadM "CITY00.BIN", 1
```